

PHASE 5

# Project Development Phase

EduGenie - AI Powered Learning Assistant

---

Record the implemented application structure, backend routes, frontend behavior, configuration, and known implementation gaps in the supplied archive.

Prepared as a phase-wise academic submission document based on the supplied EduGenie-Ai project archive.

# 1. Implemented Backend Entry Point

main.py creates a FastAPI application named EduGenie with the description "Google Gemini Powered Learning Assistant" and version 1.0.0. It mounts static files, configures Jinja2 templates, serves the root page, exposes a health endpoint, and routes the five learning tasks to dedicated modules.

# 2. Implemented API Surface

|                             |                          |                                                       |
|-----------------------------|--------------------------|-------------------------------------------------------|
| GET /                       | Home page                | main.py                                               |
| GET /health                 | Health check             | main.py                                               |
| POST /qa                    | Question answering       | main.py -> qna.answer_question                        |
| POST /explain               | Topic explanation        | main.py -> explanation_module.explain_topic           |
| POST /quiz                  | Quiz generation          | main.py -> quiz_module.generate_quiz                  |
| POST /summarize             | Text summarization       | main.py -> summary_module.summarize_text              |
| POST /learn/recommendations | Learning recommendations | main.py -> learning_path.get_learning_recommendations |

# 3. Frontend Development Evidence

- JavaScript supports five task modes: qa, explain, quiz, summarize, learn.
- The input label and placeholder are changed dynamically based on the selected task.
- A learner level selector is revealed for the learning-path task.
- Requests are sent with fetch using JSON and the Content-Type application/json header.
- The UI shows Generating... while awaiting a response and restores the button afterward.
- Generated text is escaped before insertion into the page, reducing direct HTML injection risk.
- Quiz options receive feedback based on the generated correct answer.

# 4. Styling Development Evidence

The CSS defines a responsive card-based interface with a hero area, status pill, form controls, result container, quiz question cards, path-step styling, and a mobile media query for screens up to 650px wide.

# 5. Configuration

requirements.txt specifies FastAPI, Uvicorn, Jinja2, python-dotenv, google-genai, Pydantic, httpx, and pytest. The supplied .env configures GEMINI\_MODEL as gemini-2.5-flash, temperature as 0.4, disables the local explainer flag, and names a local explainer model.

# 6. Archive Completeness Review

The supplied archive contains: main.py, requirements.txt, .env, a Jinja template path, CSS, and JavaScript UI logic. main.py defines FastAPI routes for /, /health, /qa, /explain, /quiz, /summarize, and /learn/recommendations. The JavaScript maps five user tasks to those endpoints. The archive snapshot does not include the Python modules imported by main.py (models.py, qna.py, explanation\_module.py, quiz\_module.py, summary\_module.py, learning\_path.py), and templates/index.html is empty. Therefore the documents describe the intended architecture and

the evidence visible in the supplied snapshot, while flagging these implementation gaps where relevant.

## 7. Development Actions Required Before Final Build

- Implement or restore `models.py`, `qna.py`, `explanation_module.py`, `quiz_module.py`, `summary_module.py`, and `learning_path.py`.
- Populate `templates/index.html` with the DOM elements referenced by `app.js`.
- Confirm the actual placement and inclusion of static assets used by the HTML template.
- Add tests for each route and structured response type.
- Use a fresh API credential for any final demonstration environment; do not distribute the credential shown in the supplied archive.

## 8. Development Conclusion

The supplied snapshot demonstrates the intended application backbone and client behavior, but it is not sufficient by itself for an end-to-end execution. The missing modules and empty template should be treated as mandatory development completion items.